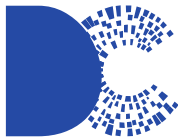




UNIVERSIDADE
FEDERAL DE
SERGIPE



DEPARTAMENTO
DE COMPUTAÇÃO

Sistema operacional de tempo real

Interface Hardware/Software

Bruno Prado

Departamento de Computação / UFS

Introdução

- ▶ O que é uma aplicação de tempo real?
 - ▶ A funcionalidade desempenhada depende do tempo de execução (*deadline*) e não somente da corretude lógica

Introdução

- ▶ O que é uma aplicação de tempo real?
 - ▶ A funcionalidade desempenhada depende do tempo de execução (*deadline*) e não somente da corretude lógica
 - ▶ Suave (soft): o atraso no prazo não deixa o sistema inoperante (lentidão em sistema multimídia)

Introdução

- ▶ O que é uma aplicação de tempo real?
 - ▶ A funcionalidade desempenhada depende do tempo de execução (*deadline*) e não somente da corretude lógica
 - ▶ Suave (*soft*): o atraso no prazo não deixa o sistema inoperante (lentidão em sistema multimídia)
 - ▶ Firme (*firm*): resultados não úteis após o *deadline* (avisar falta de combustível após falha)

Introdução

- ▶ O que é uma aplicação de tempo real?
 - ▶ A funcionalidade desempenhada depende do tempo de execução (*deadline*) e não somente da corretude lógica
 - ▶ Suave (*soft*): o atraso no prazo não deixa o sistema inoperante (lentidão em sistema multimídia)
 - ▶ Firme (*firm*): resultados não úteis após o *deadline* (avisar falta de combustível após falha)
 - ▶ Rígida (*hard*): impacto catastrófico no sistema (não acionamento do *airbag* durante colisão)

Introdução

- ▶ O que é uma aplicação de tempo real?
 - ▶ A funcionalidade desempenhada depende do tempo de execução (*deadline*) e não somente da corretude lógica
 - ▶ Suave (*soft*): o atraso no prazo não deixa o sistema inoperante (lentidão em sistema multimídia)
 - ▶ Firme (*firm*): resultados não úteis após o *deadline* (avisar falta de combustível após falha)
 - ▶ Rígida (*hard*): impacto catastrófico no sistema (não acionamento do *airbag* durante colisão)

Como satisfazer estas restrições?

Introdução

- ▶ Sistema operacional de tempo real (RTOS)
 - ▶ Define como as tarefas que serão escalonadas com diferentes níveis de prioridade e restrições de tempo

SOFT $\longleftrightarrow$ $\downarrow$ PRIORIDADE

Introdução

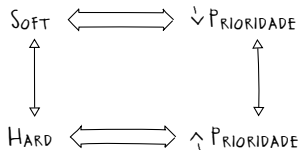
- ▶ Sistema operacional de tempo real (RTOS)
 - ▶ Define como as tarefas que serão escalonadas com diferentes níveis de prioridade e restrições de tempo

SOFT $\longleftrightarrow$ $\downarrow$ PRIORIDADE

HARD $\longleftrightarrow$ $\uparrow$ PRIORIDADE

Introdução

- ▶ Sistema operacional de tempo real (RTOS)
 - ▶ Define como as tarefas que serão escalonadas com diferentes níveis de prioridade e restrições de tempo



PROBLEMA RESOLVIDO?

Introdução

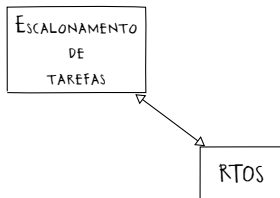
- ▶ Sistema operacional de tempo real (RTOS)
 - ▶ Define como as tarefas que serão escalonadas com diferentes níveis de prioridade e restrições de tempo



RTOS

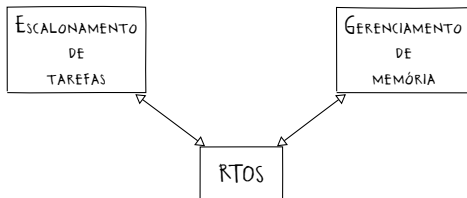
Introdução

- ▶ Sistema operacional de tempo real (RTOS)
 - ▶ Define como as tarefas que serão escalonadas com diferentes níveis de prioridade e restrições de tempo



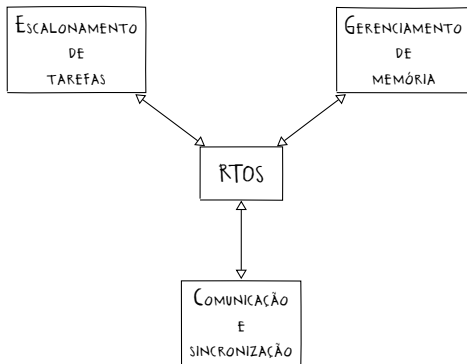
Introdução

- ▶ Sistema operacional de tempo real (RTOS)
 - ▶ Define como as tarefas que serão escalonadas com diferentes níveis de prioridade e restrições de tempo



Introdução

- ▶ Sistema operacional de tempo real (RTOS)
 - ▶ Define como as tarefas que serão escalonadas com diferentes níveis de prioridade e restrições de tempo



Introdução

- ▶ Por que utilizar um RTOS?
 - ▶ Sistema complexo
 - ▶ Execução determinística
 - ▶ Abstração de eventos e de tempo
 - ▶ Gerenciamento de recursos da plataforma

Introdução

- ▶ Por que utilizar um RTOS?
 - ▶ Sistema complexo
 - ▶ Execução determinística
 - ▶ Abstração de eventos e de tempo
 - ▶ Gerenciamento de recursos da plataforma
 - ▶ Manutenção e extensibilidade
 - ▶ API + Modularidade + Reuso de código

Introdução

- ▶ Por que utilizar um RTOS?
 - ▶ Sistema complexo
 - ▶ Execução determinística
 - ▶ Abstração de eventos e de tempo
 - ▶ Gerenciamento de recursos da plataforma
 - ▶ Manutenção e extensibilidade
 - ▶ API + Modularidade + Reuso de código
 - ▶ Gerenciamento de potência
 - ▶ Maior eficiência + Aproveitamento do tempo ocioso

Introdução

- ▶ RTOS de código aberto
 - ▶ Apache Mynewt
 - ▶ Contiki
 - ▶ eCos
 - ▶ FreeRTOS
 - ▶ Zephyr
 - ▶ ...

Introdução

- ▶ Por que usar o FreeRTOS?
 - ▶ Desenvolvido para aplicações industriais

Introdução

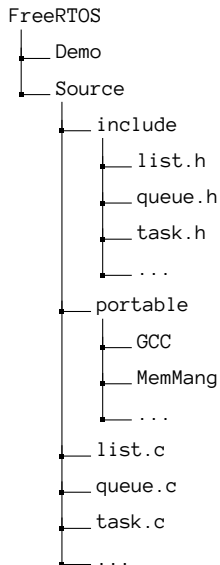
- ▶ Por que usar o FreeRTOS?
 - ▶ Desenvolvido para aplicações industriais
 - ▶ Código aberto e gratuito, permitindo uso comercial

Introdução

- ▶ Por que usar o FreeRTOS?
 - ▶ Desenvolvido para aplicações industriais
 - ▶ Código aberto e gratuito, permitindo uso comercial
 - ▶ Estrutura simples e com baixo uso de recursos

Introdução

► Estrutura de diretório



Gerenciamento de memória

- ▶ Alocação de memória
 - ▶ A memória necessária para núcleo do RTOS pode ser alocado estaticamente em tempo de compilação ou dinamicamente em tempo de execução

Gerenciamento de memória

► Alocação de memória

- A memória necessária para núcleo do RTOS pode ser alocado estaticamente em tempo de compilação ou dinamicamente em tempo de execução

Estática	Dinâmica
Alocação pré-definida pelo programador	Alocação automática com diferentes esquemas
Definição prévia do tamanho máximo da memória	A memória utilizada depende da demanda do sistema
Uso fixo da memória	Reuso de memória

Gerenciamento de memória

- ▶ Alocação dinâmica de memória
 - ▶ Apesar de suportar a alocação estática, com a alocação de memória em tempo de execução a memória é utilizada e liberada sob demanda

Gerenciamento de memória

- ▶ Alocação dinâmica de memória
 - ▶ Apesar de suportar a alocação estática, com a alocação de memória em tempo de execução a memória é utilizada e liberada sob demanda
 - ▶ Esforço menor para planejamento e projeto
 - ▶ API simplificada com menos argumentos
 - ▶ Tamanho total de memória reduzido

Gerenciamento de memória

- ▶ Alocação dinâmica de memória
 - ▶ Apesar de suportar a alocação estática, com a alocação de memória em tempo de execução a memória é utilizada e liberada sob demanda
 - ▶ Esforço menor para planejamento e projeto
 - ▶ API simplificada com menos argumentos
 - ▶ Tamanho total de memória reduzido
 - ▶ Funções *malloc* e *free* de C

Gerenciamento de memória

- ▶ Alocação dinâmica de memória
 - ▶ Apesar de suportar a alocação estática, com a alocação de memória em tempo de execução a memória é utilizada e liberada sob demanda
 - ▶ Esforço menor para planejamento e projeto
 - ▶ API simplificada com menos argumentos
 - ▶ Tamanho total de memória reduzido
 - ▶ Funções *malloc* e *free* de C
 - ▶ Não estão disponíveis em todas as plataformas
 - ▶ Geralmente utilizam muito espaço em memória
 - ▶ Dificilmente são *thread-safe* (livre de condições de corrida entre as tarefas) e determinísticas (o tempo de execução das chamadas é variável)
 - ▶ Podem gerar fragmentação de memória

Gerenciamento de memória

► Alocação dinâmica de memória

► *malloc* × *pvPortMalloc*

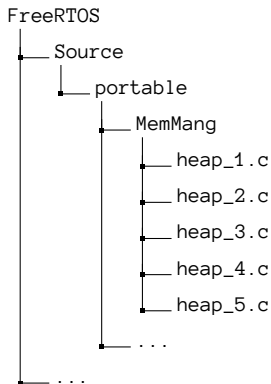
```
1  /**
2   *  Alocação dinâmica de memória
3   *  @param size Quantidade de bytes
4   */
5  void* malloc(size_t size);
6  void* pvPortMalloc(size_t size);
```

► *free* × *vPortFree*

```
1  /**
2   *  Liberação de memória alocada
3   *  @param ptr Ponteiro de memória
4   */
5  void free(void* ptr);
6  void vPortFree(void* ptr);
```

Gerenciamento de memória

- ▶ Alocação dinâmica de memória
 - ▶ Esquemas de *heap*



Gerenciamento de memória

- ▶ Esquema de heap_1
 - ▶ A alocação dinâmica de memória é realizada antes do início da execução das tarefas
 - ▶ Versão básica do *pvPortMalloc*

Gerenciamento de memória

- ▶ Esquema de heap_1
 - ▶ A alocação dinâmica de memória é realizada antes do início da execução das tarefas
 - ▶ Versão básica do *pvPortMalloc*
 - ▶ Toda memória alocada é mantida durante a execução da aplicação
 - ▶ Não implementa *vPortFree*

Gerenciamento de memória

- ▶ Esquema de heap_1
 - ▶ A alocação dinâmica de memória é realizada antes do início da execução das tarefas
 - ▶ Versão básica do *pvPortMalloc*
 - ▶ Toda memória alocada é mantida durante a execução da aplicação
 - ▶ Não implementa *vPortFree*
 - ▶ Este esquema não enfrenta problemas de determinismo e de fragmentação das operações de alocação de memória

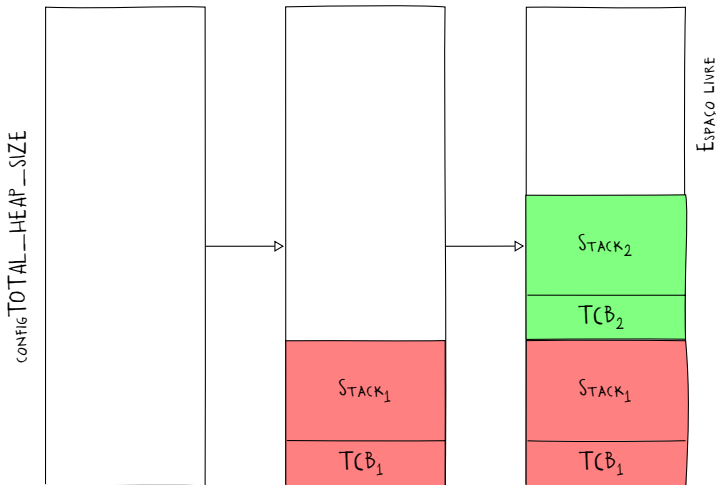
Gerenciamento de memória

- ▶ Esquema de heap_1
 - ▶ A alocação dinâmica de memória é realizada antes do início da execução das tarefas
 - ▶ Versão básica do *pvPortMalloc*
 - ▶ Toda memória alocada é mantida durante a execução da aplicação
 - ▶ Não implementa *vPortFree*
 - ▶ Este esquema não enfrenta problemas de determinismo e de fragmentação das operações de alocação de memória

Alguns sistemas críticos proíbem
a alocação dinâmica durante a execução

Gerenciamento de memória

- ▶ Esquema de heap_1
 - ▶ Tamanho total em bytes (*configTOTAL_HEAP_SIZE*)
 - ▶ *Task Control Block* (TCB) e pilha da tarefa (Stack)



Gerenciamento de memória

- ▶ Esquema de heap_2
 - ▶ É mantido para compatibilidade reversa (*deprecated*) e não deve ser utilizado em novos projetos (deve ser utilizado o esquema heap_4)

Gerenciamento de memória

- ▶ Esquema de heap_2
 - ▶ É mantido para compatibilidade reversa (*deprecated*) e não deve ser utilizado em novos projetos (deve ser utilizado o esquema heap_4)
 - ▶ Permite que a memória seja desalocada, mas não combina os blocos adjacentes livres e não possui um tempo de execução determinístico

Gerenciamento de memória

- ▶ Esquema de heap_2
 - ▶ É mantido para compatibilidade reversa (*deprecated*) e não deve ser utilizado em novos projetos (deve ser utilizado o esquema heap_4)
 - ▶ Permite que a memória seja desalocada, mas não combina os blocos adjacentes livres e não possui um tempo de execução determinístico
 - ▶ Na alocação é escolhido o bloco livre que possui o tamanho mais próximo do valor solicitado
 - ▶ Ex: blocos livres com 8, 16 e 32 bytes
 - ▶ Para uma solicitação de 14 bytes é utilizado o bloco com 16 bytes e um bloco livre com 2 bytes é criado

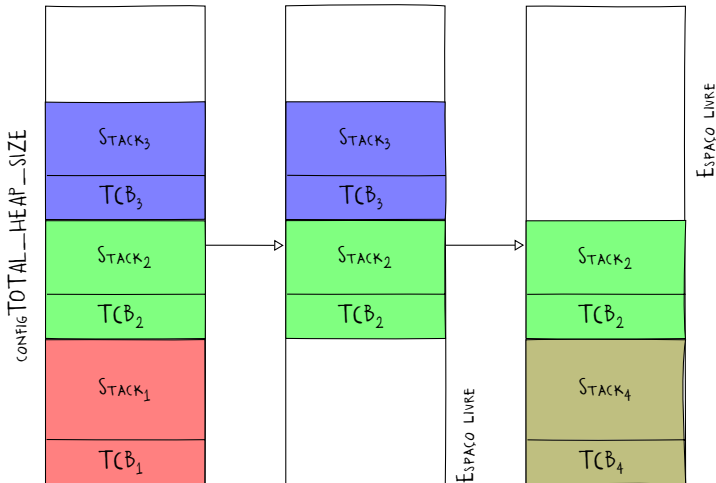
Gerenciamento de memória

- ▶ Esquema de heap_2
 - ▶ É mantido para compatibilidade reversa (*deprecated*) e não deve ser utilizado em novos projetos (deve ser utilizado o esquema heap_4)
 - ▶ Permite que a memória seja desalocada, mas não combina os blocos adjacentes livres e não possui um tempo de execução determinístico
 - ▶ Na alocação é escolhido o bloco livre que possui o tamanho mais próximo do valor solicitado
 - ▶ Ex: blocos livres com 8, 16 e 32 bytes
 - ▶ Para uma solicitação de 14 bytes é utilizado o bloco com 16 bytes e um bloco livre com 2 bytes é criado

A pilha alocada para as tarefas
devem possuir o mesmo tamanho

Gerenciamento de memória

- ▶ Esquema de heap_2
 - ▶ Tamanho total em bytes (*configTOTAL_HEAP_SIZE*)
 - ▶ *Task Control Block* (TCB) e pilha da tarefa (Stack)



Gerenciamento de memória

- ▶ Esquema de heap_3
 - ▶ Utiliza as funções padrão de C para alocação (*malloc*) e liberação (*free*) de memória

Gerenciamento de memória

- ▶ Esquema de heap_3
 - ▶ Utiliza as funções padrão de C para alocação (*malloc*) e liberação (*free*) de memória
 - ▶ O tamanho do script de linkagem, deixando a definição `configTOTAL_HEAP_SIZE` sem efeito

Gerenciamento de memória

- ▶ Esquema de heap_3
 - ▶ Utiliza as funções padrão de C para alocação (*malloc*) e liberação (*free*) de memória
 - ▶ O tamanho do script de linkagem, deixando a definição `configTOTAL_HEAP_SIZE` sem efeito
 - ▶ Através da suspensão do escalonador, as funções *malloc* e *free* se tornam *thread-safe*

Gerenciamento de memória

- ▶ Esquema de heap_4
 - ▶ Assim como nos esquemas heap_1 e heap_2, a memória é subdividida em blocos, entretanto, os blocos livres que são adjacentes são combinados para formarem um bloco único (minimiza o risco de fragmentação da memória)

Gerenciamento de memória

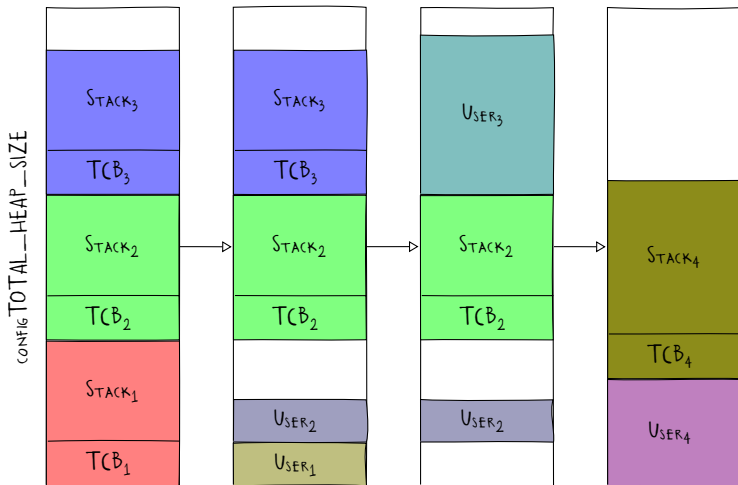
- ▶ Esquema de heap_4
 - ▶ Assim como nos esquemas heap_1 e heap_2, a memória é subdividida em blocos, entretanto, os blocos livres que são adjacentes são combinados para formarem um bloco único (minimiza o risco de fragmentação da memória)
 - ▶ O primeiro bloco que possui tamanho suficiente é alocado pela requisição (*pvPortMalloc*), logo, o tempo para alocação não é determinístico

Gerenciamento de memória

- ▶ Esquema de heap_4
 - ▶ Assim como nos esquemas heap_1 e heap_2, a memória é subdividida em blocos, entretanto, os blocos livres que são adjacentes são combinados para formarem um bloco único (minimiza o risco de fragmentação da memória)
 - ▶ O primeiro bloco que possui tamanho suficiente é alocado pela requisição (*pvPortMalloc*), logo, o tempo para alocação não é determinístico
 - ▶ É ideal para aplicações que repetidamente alocam e desalocam dados com diferentes tamanhos

Gerenciamento de memória

- ▶ Esquema de heap_4
 - ▶ Tamanho total em bytes (*configTOTAL_HEAP_SIZE*)
 - ▶ *Task Control Block* (TCB) e pilha da tarefa (Stack)



Gerenciamento de memória

► Esquema de heap_5

- O funcionamento deste esquema é idêntico ao heap_4, a única mudança está possibilidade de utilização de múltiplos espaços de endereço de memória para a alocação dos dados

```
1 // Estrutura de heap
2 typedef struct HeapRegion {
3     // Endereço inicial
4     uint8_t* pucStartAddress;
5     // Tamanho em bytes
6     size_t xSizeInBytes;
7 } HeapRegion_t;
8 /**
9  * Define os espaços de memória
10  * @param pxHeapRegions Vetor de heaps
11  */
12 void vPortDefineHeapRegions(const HeapRegion_t* const
    pxHeapRegions);
```

Gerenciamento de memória

► Esquema de heap_5

- O funcionamento deste esquema é idêntico ao heap_4, exceto pela possibilidade de utilização de múltiplos espaços de endereço para a alocação

```
1 // Definições de endereços e tamanhos
2 #define RAM1_START ((uint8_t*)(0x00010000))
3 #define RAM1_SIZE (64 * 1024)
4 #define RAM2_START ((uint8_t*)(0x00020000))
5 #define RAM2_SIZE (32 * 1024)
6 // Criação de vetor de heaps
7 const HeapRegion_t myHeapRegions[] = {
8     { RAM1_START, RAM1_SIZE },
9     { RAM2_START, RAM2_SIZE },
10    { NULL, 0 }
11 };
12 // Função principal
13 void main() {
14     // Definindo regiões de heap
15     vPortDefineHeapRegions(myHeapRegions);
16 }
```


Exercício

- ▶ Aprofunde seus conhecimentos sobre gerenciamento de memória no FreeRTOS
 - ▶ Verifique a API completa e o funcionamento dos *hook*
 - ▶ Cite exemplos de aplicações para cada um dos tipos de esquema de *heap* vistos
 - ▶ Estude a unidade de proteção de memória (MPU)

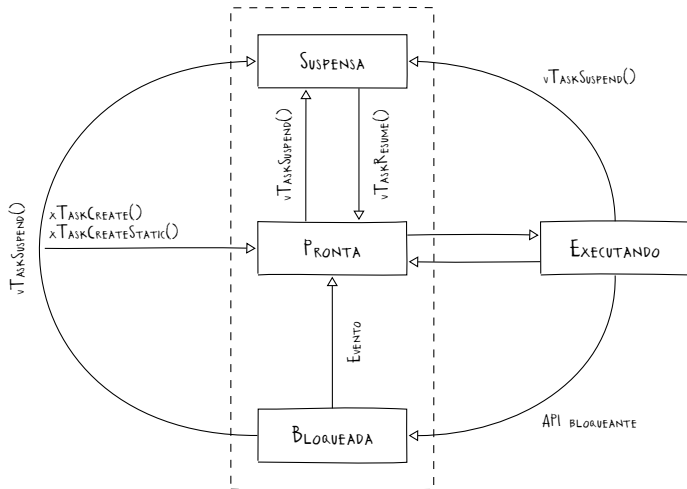
Escalonamento de tarefas

- Implementação de tarefa
 - Cada tarefa é um fluxo independente com seu próprio contexto (TCB) e parâmetros de execução

```
1 // FreeRTOS task
2 #include <task.h>
3 /**
4  * Tarefa do RTOS
5  * @param pvParameters Parâmetros da tarefa
6  */
7 void tarefa(void* pvParameters) {
8     // Variável local
9     uint32_t local = 0;
10    // Loop infinito
11    while(1) {
12        // Operações da tarefa
13        ...
14    }
15    // Remove a tarefa
16    vTaskDelete(NULL);
17 }
```

Escalonamento de tarefas

- ▶ Máquina de estado das tarefas
 - ▶ Não Executando × Executando



Escalonamento de tarefas

► Criação de tarefa estática

```
1  /**
2   *   Criar tarefa estática
3   *   @param pxTaskCode      Ponteiro para função
4   *   @param pcName          Descrição em texto
5   *   @param ulStackDepth    Tamanho da pilha
6   *   @param pvParameters    Parâmetros da tarefa
7   *   @param uxPriority       Prioridade de execução
8   *   @param puxStackBuffer   Ponteiro para pilha
9   *   @param pxTaskBuffer     Ponteiro para dados
10  *   @return                  É retornado o manipulador
    da tarefa (sucesso) ou ponteiro nulo (erro)
11  */
12  TaskHandle_t xTaskCreateStatic(TaskFunction_t
    pxTaskCode, const char* const pcName, const
    uint32_t ulStackDepth, void* const pvParameters,
    UBaseType_t uxPriority, StackType_t* const
    puxStackBuffer, StaticTask_t* const
    pxTaskBuffer);
```

Escalonamento de tarefas

► Criação de tarefa estática

```
1 // Pilha com 200 bytes
2 #define STACK_SIZE 200
3 StackType_t xStack[STACK_SIZE];
4 // Variáveis da tarefa
5 StaticTask_t xTaskBuffer;
6 TaskHandle_t xHandle = NULL;
7 // Função da tarefa
8 void tarefa(void* pvParameters) {
9     // Conversão de tipo de parâmetro
10    uint32_t p = *(uint32_t*)(pvParameters);
11    // Laço infinito
12    while(1) {
13        // Incremento de variável
14        p++;
15        // Suspensão de tarefa
16        taskYIELD();
17    }
18 }
... ..
```

Escalonamento de tarefas

► Criação de tarefa estática

```
1  // Pilha com 200 bytes
2  #define STACK_SIZE 200
3  StackType_t xStack[STACK_SIZE];
4  // Variáveis da tarefa
5  StaticTask_t xTaskBuffer;
6  TaskHandle_t xHandle = NULL;
...
19 // Função criador
20 void criador() {
21     // Criando tarefa estática
22     xHandle = xTaskCreateStatic(tarefa,
        "MinhaTarefa", STACK_SIZE, (void*)(1),
        tskIDLE_PRIORITY, xStack, &xTaskBuffer);
23 }
```

Escalonamento de tarefas

► Criação de tarefa dinâmica

```
1  /**
2   *   Criar tarefa
3   *   @param pxTaskCode      Ponteiro para função
4   *   @param pcName          Descrição em texto
5   *   @param usStackDepth    No. de palavras da pilha
6   *   @param pvParameters    Parâmetros da tarefa
7   *   @param uxPriority       Prioridade de execução
8   *   @param pxCreatedTask    Ponteiro para manipulador
9   *   @return                 Retorna pdPASS (sucesso)
10   *                           ou errCOULD_NOT_ALLOCATE_REQUIRED_MEMORY (erro)
11  */
BaseType_t xTaskCreate(TaskFunction_t pxTaskCode,
    const char* const pcName, configSTACK_DEPTH_TYPE
    usStackDepth, void* pvParameters, UBaseType_t
    uxPriority, TaskHandle_t* pxCreatedTask);
```

Escalonamento de tarefas

► Criação de tarefa dinâmica

```
1 // Pilha com 200 bytes
2 #define STACK_SIZE 200
3 // Manipulador da tarefa
4 TaskHandle_t xHandle = NULL;
5 // Função da tarefa
6 void tarefa(void* pvParameters) {
7     // Conversão de tipo de parâmetro
8     uint32_t p = *(uint32_t*)(pvParameters);
9     // Atraso de tempo de 250 ms
10    const TickType_t t250ms = pdMS_TO_TICKS(250);
11    // Laço infinito
12    while(1) {
13        // Incremento de variável
14        p++;
15        // Suspensão de tarefa por 250 ms
16        vTaskDelay(t250ms);
17    }
18 }
... ..
```


Escalonamento de tarefas

► Criação de tarefa dinâmica

```
1 // Pilha com 200 bytes
2 #define STACK_SIZE 200
3 // Manipulador da tarefa
4 TaskHandle_t xHandle = NULL;
...
19 // Função criador
20 void criador(TaskHandle_t* xHandle) {
21     // Criando tarefa dinamicamente
22     BaseType_t xReturned = xTaskCreate(tarefa,
23         "MinhaTarefa", STACK_SIZE, (void*)(1),
24         tskIDLE_PRIORITY, &xHandle);
25 }
```

Escalonamento de tarefas

► Prioridade para escalonamento das tarefas

```
1 // Descrição das tarefas
2 static const char* m1 = "Tarefa_1!\n";
3 static const char* m2 = "Tarefa_2!\n";
4 // Função da tarefa
5 void T(void* pvParameters) {
6     // Comportamento da tarefa
7     printf("%s\n", (char*)(pvParameters));
8     while(1);
9 }
10 // Função principal
11 void main() {
12     // Prioridade de 0 (menor) até configMAX_PRIORITIES
13     // - 1 (maior)
14     xTaskCreate(T, "T1", 512, (void*)(m1), 1, NULL);
15     xTaskCreate(T, "T2", 512, (void*)(m2), 2, NULL);
16     // Inicialização do escalonador
17     vTaskStartScheduler();
18 }
```

Escalonamento de tarefas

► Prioridade para escalonamento das tarefas

```
1 // Descrição das tarefas
2 static const char* m1 = "Tarefa_1!\n";
3 static const char* m2 = "Tarefa_2!\n";
4 // Função da tarefa
5 void T(void* pvParameters) {
6     // Comportamento da tarefa
7     printf("%s\n", (char*)(pvParameters));
8     while(1);
9 }
10 // Função principal
11 void main() {
12     // Prioridade de 0 (menor) até configMAX_PRIORITIES
13     // - 1 (maior)
14     xTaskCreate(T, "T1", 512, (void*)(m1), 1, NULL);
15     xTaskCreate(T, "T2", 512, (void*)(m2), 2, NULL);
16     // Inicialização do escalonador
17     vTaskStartScheduler();
18 }
```

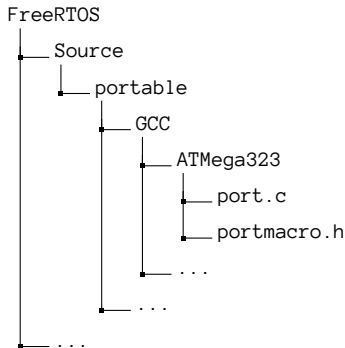
A tarefa "T1" nunca irá executar (*starvation*)

Exercício

- ▶ Estude e pesquise sobre as facilidades do FreeRTOS
 - ▶ Gerenciamento de filas
 - ▶ Temporizadores
 - ▶ Gerenciamento de interrupção
 - ▶ Notificações e sincronização
 - ▶ Suporte para depuração

Dependências da plataforma

- ▶ Código fonte dependente da plataforma
 - ▶ Compilador GCC
 - ▶ ATMega323



Dependências da plataforma

- ▶ Arquivo fonte portmacro.h
 - ▶ Definições da seção crítica e interrupção

```
1 // Entrando em região crítica
2 #define portENTER_CRITICAL() \
3     asm volatile("in_%%tmp_reg_, %%SREG_" ::); \
4     asm volatile("cli" ::); \
5     asm volatile("push_%%tmp_reg_" ::)
6 // Saindo de região crítica
7 #define portEXIT_CRITICAL() \
8     asm volatile("pop_%%tmp_reg_" ::); \
9     asm volatile("out_%%SREG_, %%tmp_reg_" ::)
10 // Desabilitando interrupções
11 #define portDISABLE_INTERRUPTS() \
12     asm volatile("cli" ::);
13 // Habilitando interrupções
14 #define portENABLE_INTERRUPTS() \
15     asm volatile("sei" ::);
```

Dependências da plataforma

- ▶ Arquivo fonte portmacro.h
 - ▶ Definições da arquitetura

```
1 // Direção da pilha
2 #define portSTACK_GROWTH    (-1)
3 // Período de tempo de escalonamento (tick)
4 #define portTICK_PERIOD_MS \
5     ((TickType_t)(1000) / configTICK_RATE_HZ)
6 // Alinhamento de byte
7 #define portBYTE_ALIGNMENT (1)
8 // Instrução nop
9 #define portNOP() \
10     asm volatile ("nop");
```

Dependências da plataforma

► Arquivo fonte port.c

```
1 // Função de inicialização da pilha
2 StackType_t* pxPortInitialiseStack(StackType_t*
    pxTopOfStack, TaskFunction_t pxCode, void*
    pvParameters) {
3     uint16_t usAddress;
4     ...
21    usAddress = (uint16_t)(pxCode);
22    *pxTopOfStack = (StackType_t)(usAddress &
        (uint16_t)(0x00FF));
23    pxTopOfStack--;
24    usAddress >>= 8;
25    *pxTopOfStack = (StackType_t)(usAddress &
        (uint16_t)(0x00FF));
26    pxTopOfStack--;
27    ...
33    *pxTopOfStack = (StackType_t)(0x00); // R0
34    pxTopOfStack--;
35    ...
109 }
```


Dependências da plataforma

► Arquivo fonte port.c

```
1 // Função de inicialização da pilha
2 StackType_t* pxPortInitialiseStack(StackType_t*
    pxTopOfStack, TaskFunction_t pxCode, void*
    pvParameters) {
...
    ...
84 *pxTopOfStack = (StackType_t)(0x23); // R23
85 pxTopOfStack--;
86 usAddress = (uint16_t)(pvParameters);
87 *pxTopOfStack = (StackType_t)(usAddress &
    (uint16_t)(0x00FF));
88 pxTopOfStack--; usAddress >>= 8;
89 *pxTopOfStack = (StackType_t)(usAddress &
    (uint16_t)(0x00FF));
90 pxTopOfStack--;
...
    ...
106 *pxTopOfStack = (StackType_t)(0x31); // R31
107 pxTopOfStack--;
108 return pxTopOfStack;
109 }
```

Dependências da plataforma

► Arquivo fonte port.c

```
1 // Salvando o contexto
2 #define portSAVE_CONTEXT() \
3     asm volatile(\
4         "push_r0_\n\t"\
5         "in_r0, _SREG_\n\t"\
6         "cli_\n\t"\
7         "push_r0_\n\t"\
8         ...
37        "push_r31_\n\t"\
38        "lds_r26, _pxCurrentTCB_\n\t"\
39        "lds_r27, _pxCurrentTCB_+1_\n\t"\
40        "in_r0, _0x3d_\n\t"\
41        "st_x+, r0_\n\t"\
42        "in_r0, _0x3e_\n\t"\
43        "st_x+, r0_\n\t"\
44    );
```

Dependências da plataforma

► Arquivo fonte port.c

```
1 // Salvando o contexto
2 #define portSAVE_CONTEXT() \
3     asm volatile(\
4         "push_r0_\n\t"\
5         "in_r0, _SREG_\n\t"\
6         "cli_\n\t"\
7         "push_r0_\n\t"\
8         ...
37        "push_r31_\n\t"\
38        "lds_r26, _pxCurrentTCB_\n\t"\
39        "lds_r27, _pxCurrentTCB_+1_\n\t"\
40        "in_r0, _0x3d_\n\t"\
41        "st_x+, r0_\n\t"\
42        "in_r0, _0x3e_\n\t"\
43        "st_x+, r0_\n\t"\
44    );
```

Salvando registradores na pilha

Dependências da plataforma

► Arquivo fonte port.c

```
1 // Salvando o contexto
2 #define portSAVE_CONTEXT() \
3     asm volatile(\
4         "push_r0\n\t"\
5         "in_r0, _SREG_\n\t"\
6         "cli\n\t"\
7         "push_r0\n\t"\
8         ...
37        "push_r31\n\t"\
38        "lds_r26, _pxCurrentTCB\n\t"\
39        "lds_r27, _pxCurrentTCB+1\n\t"\
40        "in_r0, 0x3d\n\t"\
41        "st_x+, r0\n\t"\
42        "in_r0, 0x3e\n\t"\
43        "st_x+, r0\n\t"\
44    );
```

Carregando ponteiro da tarefa atual

Dependências da plataforma

► Arquivo fonte port.c

```
1 // Salvando o contexto
2 #define portSAVE_CONTEXT() \
3     asm volatile(\
4         "push_r0_\n\t"\
5         "in_r0, __SREG__\n\t"\
6         "cli_\n\t"\
7         "push_r0_\n\t"\
8         ...
37        "push_r31_\n\t"\
38        "lds_r26, _pxCurrentTCB_\n\t"\
39        "lds_r27, _pxCurrentTCB_+1_\n\t"\
40        "in_r0, 0x3d_\n\t"\
41        "st_x+, r0_\n\t"\
42        "in_r0, 0x3e_\n\t"\
43        "st_x+, r0_\n\t"\
44    );
```

Salvando topo da pilha em x
(SPL = 0x3D e SPH = 0x3E)

Dependências da plataforma

► Arquivo fonte port.c

```
1 // Restaurando o contexto
2 #define portRESTORE_CONTEXT() \
3     asm volatile(\
4         "lds_r26, _pxCurrentTCB_\n\t"\
5         "lds_r27, _pxCurrentTCB_+1_\n\t"\
6         "ld_r28, _x+\n\t"\
7         "out_ __SP_L__, r28\n\t"\
8         "ld_r29, _x+\n\t"\
9         "out_ __SP_H__, r29\n\t"\
10        "pop_r31\n\t"\
11        ...
39        "pop_r0\n\t"\
40        "out_ __SREG__, r0\n\t"\
41        "pop_r0\n\t"\
42    );
```

Dependências da plataforma

► Arquivo fonte port.c

```
1 // Restaurando o contexto
2 #define portRESTORE_CONTEXT() \
3     asm volatile(\
4         "lds_r26, _pxCurrentTCB_0\n\t"\
5         "lds_r27, _pxCurrentTCB_1\n\t"\
6         "ld_r28, _x+0\n\t"\
7         "out_0__SP_L__, r28\n\t"\
8         "ld_r29, _x+0\n\t"\
9         "out_0__SP_H__, r29\n\t"\
10        "pop_r31\n\t"\
11        ...
39        "pop_r0\n\t"\
40        "out_0__SREG__, r0\n\t"\
41        "pop_r0\n\t"\
42    );
```

Carregando ponteiro da tarefa atual

Dependências da plataforma

► Arquivo fonte port.c

```
1 // Restaurando o contexto
2 #define portRESTORE_CONTEXT() \
3     asm volatile(\
4         "lds_r26, _pxCurrentTCB_\\n\\t" \
5         "lds_r27, _pxCurrentTCB_+1_\\n\\t" \
6         "ld_r28, _x+\\n\\t" \
7         "out_--SP_L_, r28\\n\\t" \
8         "ld_r29, _x+\\n\\t" \
9         "out_--SP_H_, r29\\n\\t" \
10        "pop_r31_\\n\\t" \
11        ...
39        "pop_r0_\\n\\t" \
40        "out_--SREG_, r0_\\n\\t" \
41        "pop_r0_\\n\\t" \
42    );
```

O topo da pilha recebe o conteúdo do ponteiro x

Dependências da plataforma

► Arquivo fonte port.c

```
1 // Restaurando o contexto
2 #define portRESTORE_CONTEXT() \
3     asm volatile(\
4         "lds_r26, _pxCurrentTCB_0\n\t"\
5         "lds_r27, _pxCurrentTCB_1\n\t"\
6         "ld_r28, _x+0\n\t"\
7         "out_0__SP_L__, r28\n\t"\
8         "ld_r29, _x+0\n\t"\
9         "out_0__SP_H__, r29\n\t"\
10        "pop_r31\n\t"\
11        ...
39        "pop_r0\n\t"\
40        "out_0__SREG__, r0\n\t"\
41        "pop_r0\n\t"\
42    );
```

Restaurando o valor dos registradores

Dependências da plataforma

- ▶ Arquivo fonte port.c
 - ▶ Troca de contexto manual

```
1 // Protótipo de função
2 void vPortYield() __attribute__((naked));
3 // Função de suspensão de tarefa
4 void vPortYield() {
5     // Salva o contexto
6     portSAVE_CONTEXT();
7     // Realiza o escalonamento
8     vTaskSwitchContext();
9     // Restaura o contexto
10    portRESTORE_CONTEXT();
11    // Retorna da chamada (naked)
12    asm volatile("ret");
13 }
```

Dependências da plataforma

- ▶ Arquivo fonte port.c
 - ▶ Troca de contexto por tempo (*tick*)

```
1 // Protótipo de função
2 void vPortYieldFromTick() __attribute__((naked));
3 // Função de suspensão por tempo
4 void vPortYieldFromTick() {
5     // Salva o contexto
6     portSAVE_CONTEXT();
7     // Incrementa e verifica a quantidade de ticks
8     if(xTaskIncrementTick() != pdFALSE) {
9         // Realiza o escalonamento
10        vTaskSwitchContext();
11    }
12    // Restaura do contexto
13    portRESTORE_CONTEXT();
14    // Retorna da chamada (naked)
15    asm volatile("ret");
16 }
```

Preemptivo por interrupção ou escalonamento

Exercício

- ▶ Utilizando as arquiteturas disponíveis no QEMU, como ARM, AVR ou RISC-V, desenvolva e simule uma aplicação de consumidor e produtor
 - ▶ Aplique a API do RTOS para tratar a concorrência
 - ▶ O valor do contador varia de 0 até 10
 - ▶ Devem ser criadas 4 instâncias para cada tarefa de consumidor e de produtor no sistema